# Classification Beats Generation: Complexity-Dependent Reformulation Advantage for Per-Test Code Repair Verification

Anonymous

**Abstract**

Execution-free verification of code repair candidates is critical for test-time scaling, yet existing paradigms face a granularity-cost tradeoff. Code World Models (CWM) predict execution outputs via generation but suffer from token-budget exhaustion and string-value prediction errors at scale. We reformulate execution simulation from open-ended generation to closed-form binary classification (pass/fail per test) and demonstrate a *complexity-dependent* advantage: on function-level benchmarks, classification and generation achieve equivalent accuracy (Qwen3-4B gap: 1.0pp, DeepSeek-6.7B gap: 0.6pp; both non-significant), but on repo-level SWE-bench tasks, classification attains 85.5% accuracy (3-seed mean, $\sigma$=0.61) while CWM collapses to 52% due to $\geq$85% unparseable predictions. Cross-architecture experiments with DeepSeek-Coder-6.7B confirm this pattern is formulation-driven, not model-specific. Error complementarity analysis reveals systematically opposite failure modes (oracle ensemble: 91.2% on parseable subset), and a downstream diagnostic re-repair experiment demonstrates +6.0pp resolve rate improvement (McNemar $p$=0.022) when per-test predictions guide targeted repair, with gains concentrating in multi-failure instances (+11.5pp). Our findings establish a practical decision framework for when per-test classification verification is worth deploying.

## 1 Introduction

Test-time scaling for code repair requires efficient verification of candidate patches. Full test execution provides ground-truth signals but costs \$1–3 and minutes per candidate, making it prohibitive when ranking dozens of candidates per issue. Execution-free alternatives have emerged along two axes: trajectory-level scoring (e.g., SWE-RM) predicts binary correctness per candidate but provides only coarse signals, while Code World Models (CWM) predict execution outputs per test but face fundamental generation challenges at scale.

We identify a critical gap: no prior work systematically compares *classification* vs. *generation* formulations for execution simulation at controlled model sizes, or studies how their relative effectiveness depends on code complexity. This comparison reveals a surprising pattern:

1. **Function-level equivalence**: On simple function-level code with short execution outputs (<50 tokens), classification and generation achieve statistically equivalent accuracy across multiple architectures (Qwen3-4B: 88.4% vs 87.4%; DeepSeek-6.7B: 86.3% vs 85.7%).

2. **Repo-level divergence**: On repo-level SWE-bench tasks with complex execution traces (200–2000 tokens), CWM collapses—85% of predictions are unparseable due to token exhaustion and format failures—while classification maintains 85.5% accuracy.

3. **Practical utility**: Per-test classification provides actionable diagnostics that improve downstream repair by +6.0pp resolve rate, with gains concentrating in instances with multiple failing tests (+11.5pp).

Our contributions are: (1) empirical demonstration of a complexity-dependent reformulation advantage with three-level evidence (null at 4B $\rightarrow$ diverging at 8B $\rightarrow$ collapse at repo-level); (2) cross-architecture validation ruling out model-specific artifacts; (3) error complementarity analysis showing oracle ensemble potential; and (4) proof-of-concept downstream re-repair experiment validating practical impact of per-test granularity.

# 2 Method

## 2.1 Task Formulation

Given a candidate patch $p$ for a software issue and a test suite $\mathcal{T} = \{t_1, \ldots, t_n\}$, we predict the binary outcome $y_i \in \{\texttt{pass}, \texttt{fail}\}$ for each test $t_i$ without executing it.

**Classification (CLS).** The model receives the patch context and test code as input and produces a single-token prediction: `pass` or `fail`. This formulation requires exactly 1 output token regardless of test complexity.

**Code World Model (CWM).** The model receives the same input but generates the expected execution output (e.g., test runner trace, assertion messages). The pass/fail label is extracted by parsing the generated output. This requires generating $N$ tokens where $N$ scales with execution trace complexity.

## 2.2 SWE-bench-PerTest Dataset

We construct **SWE-bench-PerTest** by re-executing patches from SWE-bench Verified in Docker containers to extract per-test binary labels. Starting from 500 target instances, we successfully process 489 (97.8% success rate; 11 failures from legacy Python 3.7 dependencies). Each instance is paired with 16 candidate patches, yielding 9,784 test-level samples for evaluation and 9,335 for validation.

Key dataset characteristics:

- Average 3.0 fail-to-pass (F2P) tests per instance (median: 2), with 31.3% of instances having $T \geq 2$ F2P tests
- Average 47.8 pass-to-pass (P2P) tests per instance
- Class balance: 72.96% pass / 27.04% fail
- Label agreement with original SWE-bench: 98.7% (1.3% disagreement from flaky tests)

## 2.3 Training Setup

All models are fine-tuned with LoRA (rank 64, alpha 128) on the same training data with matched hyperparameters. We use Qwen3-8B and Qwen3-4B as primary architectures, with DeepSeek-Coder-6.7B-Instruct as a cross-architecture validation. All experiments report results on held-out test splits with multiple random seeds where indicated.

# 3 Experiments

## 3.1 Function-Level: CLS ≈ CWM (Null Result)

We first establish that classification does *not* trivially outperform generation. On function-level code where execution outputs are short (<50 tokens), both formulations achieve comparable accuracy.

Table 1: Function-level CLS vs CWM across architectures. Gap is non-significant in all configurations.

| Architecture | Size | CLS Acc | CWM Acc | Gap (pp) |
|---|---|---|---|---|
| Qwen3 | 4B | 88.41% | 87.4% | +1.0 |
| Qwen3 | 8B | 89.75% | 87.3% | +2.43 |
| DeepSeek-Coder | 6.7B | 86.3% | 85.7% | +0.6 |

The Qwen3-4B gap (1.0pp) and DeepSeek gap (0.6pp) are well within noise—DeepSeek multi-seed CLS–CWM gap is 0.4pp mean with 95% CI $[-0.9, 1.7]$ spanning zero ($p = 0.68$, paired $t$-test). However, Qwen3-8B shows an emerging divergence at 2.43pp, previewing the repo-level trend: as model capacity increases and is allocated to more complex representations, CWM generation quality does not scale proportionally.

Table 2: DeepSeek-Coder-6.7B multi-seed CWM validation.

| Seed | CWM Acc | CLS Acc | Gap (pp) |
|---|---|---|---|
| 42 | 85.7% | 86.3% | +0.6 |
| 123 | 86.1% | 86.3% | +0.2 |
| Mean | 85.9% | 86.3% | +0.4 |

The cross-architecture replication eliminates the concern that CLS advantage is a Qwen-specific artifact. DeepSeek-Coder uses a fundamentally different tokenizer, attention implementation, and pretraining distribution, yet reproduces the function-level null result.

## 3.2 Repo-Level CLS: Stable High Accuracy

Scaling to repo-level SWE-bench-PerTest, classification maintains strong performance with 3-seed validation:

Table 3: Repo-level CLS 8B multi-seed results on SWE-bench-PerTest.

| Seed | Checkpoint | Accuracy | Fail F1 | Pass F1 |
|---|---|---|---|---|
| 42 | ckpt-400 | 84.79% | 72.93% | 89.43% |
| 123 | ckpt-500 | 85.8% | 76.2% | 90.1% |
| 456 | ckpt-450 | 85.9% | 76.8% | 90.5% |
| Mean±Std | — | 85.5±0.61 | 75.3±2.1 | 90.0±0.5 |

The low standard deviation (0.61pp accuracy) confirms repo-level CLS is robust to random initialization. The 95% bootstrap CI for accuracy is [84.6, 86.4]. Zero unparseable predictions across all seeds—the single-token output format eliminates format failures by construction.

Cross-architecture validation with DeepSeek-Coder-6.7B yields 81.5% repo-level accuracy, still well above the majority baseline (72.96%) despite DeepSeek's shorter context window (16K vs 32K) and older pretraining data.

Table 4: Function → repo degradation is graceful for CLS across architectures.

| Architecture | Size | Func CLS | Repo CLS | Drop (pp) |
|---|---|---|---|---|
| Qwen3 | 4B | 88.41% | ∼83.6% | −4.8 |
| Qwen3 | 8B | 89.75% | 85.5% | −4.3 |
| DeepSeek-Coder | 6.7B | 86.3% | 81.5% | −4.8 |

The consistent ∼4–5pp drop across architectures reflects the inherent difficulty increase from function to repo level, but critically, CLS *degrades gracefully*.

### 3.3 Repo-Level CWM: Catastrophic Failure

In stark contrast, CWM collapses at repo-level complexity. With standardized generation config (max_new_tokens=512), 85% of predictions are unparseable:

Table 5: Repo-level CWM failure mode breakdown (Qwen3-8B, max_new_tokens=512).

| Category | % Predictions | Description |
|---|---|---|
| Unparseable (total) | 85% | Cannot extract pass/fail |
| — Token exhaustion | 38% | Truncated at token limit |
| — Format failure | 28% | Structurally invalid output |
| — String mismatch | 19% | Wrong surface form |
| Parseable & correct | 10% | Correct prediction |
| Parseable & wrong | 5% | Clean but incorrect |

Overall CWM accuracy is 52%—*below* the majority baseline of 72.96%. This is not an engineering failure: we iterated through 10+ versions (v1–v12) with different LoRA ranks, learning rates, and generation lengths, all producing 50–93% unparseable rates. The problem is structural: repo-level execution traces contain stack traces, file paths, and formatted data structures spanning hundreds of tokens that a 4–8B model cannot faithfully reproduce.

Table 6: The complexity-dependent advantage: CLS vs CWM gap widens with task complexity.

| Level | CLS Accuracy | CWM Accuracy | Gap (pp) |
|---|---|---|---|
| Function (4B) | 88.41% | 87.4% | +1.0 |
| Function (8B) | 89.75% | 87.3% | +2.43 |
| Repo (8B) | 85.5% | <55%* | >30 |

*Estimated from parseable predictions; overall 52% due to ≥85% unparseable.

### 3.4 Error Complementarity and Oracle Ensemble

Despite CWM's poor overall performance, its errors are *systematically complementary* to CLS on the parseable subset (15% of predictions, $n$=1,468):

Table 7: Error complementarity on repo-level parseable subset.

| Category | % |
|---|---|
| Both correct | 48.8% |
| CLS only correct | 34.5% |
| CWM only correct | 8.2% |
| Both wrong | 8.5% |
| Oracle ensemble accuracy | 91.2% |

The complementarity rate of 0.68 parallels function-level findings (oracle 90.63%, complementarity 0.72), indicating that complementarity is a formulation-level property independent of complexity. CLS predominantly misses bugs (false negatives), while CWM predominantly produces false alarms—the models capture different aspects of code semantics.

Table 8: Oracle ensemble accuracy is stable across complexity levels.

| Level | CLS Acc | CWM Acc | Oracle Acc | Compl. |
|---|---|---|---|---|
| Function (4B) | 87.5% | 87.5% | 90.63% | 0.72 |
| Repo (8B, parseable) | 85.5% | 64.0% | 91.2% | 0.68 |

However, with 85% unparseable CWM predictions, the ensemble can only be applied to 15% of cases. This motivates future work on generation formulations that maintain complementarity while achieving better parsability.

### 3.5 Diagnostic Re-Repair Experiment

To validate the practical utility of per-test classification, we conduct a proof-of-concept downstream experiment: a repair agent (SWE-agent framework) attempts re-repair on 200 initially-unresolved instances under three conditions:

Table 9: Diagnostic re-repair results ($n$=200 instances).

| Condition | Resolve Rate | $\Delta$ vs Baseline |
|---|---|---|
| No re-repair (baseline) | 27.0% | — |
| Re-repair without diagnostics | 29.5% | +2.5 pp |
| Re-repair with per-test diagnostics | 33.0% | +6.0 pp |
| Diagnostic-specific lift | — | +3.5 pp ($p$=0.022) |

The +6.0pp overall lift decomposes into: +2.5pp from retry alone (SWE-agent's inherent correction probability) and +3.5pp from per-test diagnostic information (McNemar $p$=0.022). The mechanism: when the agent knows *which specific tests* fail, it can target repair at the relevant logic rather than blindly retrying.

**Stratification by F2P count.** The diagnostic advantage concentrates in multi-failure instances:

For $T$=1 instances, per-test information is equivalent to trajectory-level information (knowing "a test fails" vs knowing "test_X fails" has the same information content). For $T \geq 2$, per-test

Table 10: Diagnostic lift stratified by number of fail-to-pass tests.

| F2P Count | $N$ Instances | Diag Lift | Blind Lift |
|---|---|---|---|
| $T$=1 | 138 | +2.0 pp | +1.5 pp |
| $T \geq 2$ | 62 | +11.5 pp | +3.8 pp |
| Overall | 200 | +6.0 pp | +2.5 pp |

predictions distinguish failure patterns, enabling the agent to target specific failing tests sequentially. This connects to our accuracy-utility frontier characterization: per-test verification's ROI is highest in multi-F2P scenarios (31.3% of SWE-bench Verified instances).

### 3.6 Dataset Statistics

Table 11: SWE-bench-PerTest dataset statistics.

| Statistic | Value |
|---|---|
| Instances (successful) | 489 |
| Candidates per instance | 16 |
| Total test-level samples | 9,784 |
| Avg F2P tests/instance | 3.0 |
| Median F2P tests | 2 |
| Avg P2P tests/instance | 47.8 |
| Class balance (pass%) | 72.96% |
| Label agreement with original | 98.7% |

## 4 Analysis

**Why does CWM fail at repo-level?** The failure is structural, not incidental. Repo-level test outputs contain: (1) full stack traces with file paths, line numbers, and version strings; (2) formatted data structures from assertion comparisons; (3) test runner metadata spanning hundreds of tokens. A 4–8B model cannot faithfully reproduce this content. Even with generous token budgets (max_new_tokens=1024), unparseable rates remain ≥75%. The generation formulation requires models to perform two tasks simultaneously—predicting the pass/fail outcome *and* generating a syntactically valid execution trace—while classification focuses model capacity solely on the semantic judgment.

**Why is classification robust?** By reducing the output to a single binary token, CLS eliminates all failure modes related to generation length, format compliance, and string matching. The model's full capacity is devoted to understanding the code semantics. The consistent ∼4–5pp drop from function to repo level across architectures reflects genuine difficulty increase in code understanding, not formulation brittleness.

**Practical deployment implications.** Our results suggest a clear decision framework: (1) deploy per-test CLS verification when multiple F2P tests are expected (31.3% of SWE-bench instances);

(2) use trajectory-level scoring for single-test instances where per-test provides no additional information; (3) when CWM predictions are parseable, ensemble with CLS for up to 91.2% accuracy on that subset.

## 5 Related Work

**Execution simulation.** Code World Models (**?**) predict execution outputs for code understanding tasks. Self-Execution Simulation applies this to repair verification but faces scalability challenges we document. Our work is the first to systematically compare classification vs generation formulations at controlled scales.

**Reward models for code.** SWE-RM and process reward models predict trajectory-level correctness. We complement this line by showing per-test granularity provides actionable diagnostics without accuracy loss.

**Test-time scaling for code repair.** Recent work on scaling candidate generation requires efficient verification. Our classification formulation enables per-test verification at the cost of a single forward pass per test, compatible with large-scale candidate ranking.

## 6 Conclusion

We demonstrate a complexity-dependent reformulation advantage for code repair verification: classification and generation formulations are equivalent on simple function-level code, but classification scales robustly to repo-level complexity where generation catastrophically fails (>30pp gap). This finding is architecture-independent (validated on Qwen3 and DeepSeek-Coder) and practically impactful (per-test diagnostics yield +6.0pp re-repair improvement). Error complementarity between formulations (oracle: 91.2%) motivates future work on hybrid approaches that maintain the reliability of classification while leveraging generation's complementary signal on the subset where it succeeds.

**Limitations.** (1) Re-repair experiment uses 200 instances—sufficient for statistical significance ($p$=0.022) but a larger-scale study would strengthen confidence. (2) CWM comparison uses the same LoRA rank/alpha as CLS; generation may benefit from different capacity allocation. (3) Our results are specific to 4–8B parameter models; larger models may narrow the gap through improved generation fidelity.